

One calculation, many savings scenarios

Daily learning note | 30 September 2026 | C002

Sidiq review status: Not yet reviewed.

Research question and hypothesis

Can a reusable C++ function reproduce annual compound growth and reject invalid inputs? The hypothesis is that one explicit mathematical contract can serve both a small demonstration and independent tests. C001 established the one-year calculation last Wednesday. One year cannot distinguish simple interest from compounding, so today extends the horizon and separates calculation from display. Wednesday's C++ slot takes precedence; Q008 remains next for October 1.

Beginner explanation and definitions

Imagine an account with 1,000 invented units. At 5%, the first year's interest is 50 units. If the interest stays in the account, year two starts with 1,050 units and earns 52.50. The extra 2.50 is interest earned on earlier interest. A reusable function is a named calculation that accepts values and returns a result. The caller chooses the inputs; the function applies the same rules each time.

Principal P: opening balance. **Effective annual rate r:** growth over one model year, written as a fraction. **Horizon n:** number of whole years. **Compounding:** reinvesting earned interest. **Simple interest:** interest calculated only on the original principal. **Parameter:** named input inside a function. **Argument:** value supplied by its caller. **Return value:** the result sent back. **NaN:** a floating-point value meaning not a number. **Overflow:** a calculation exceeds the numeric range. **Exception:** an explicit error that a caller can catch rather than treating it as a balance.

Data, metrics and predeclared protocol

All balances, rates and horizons are synthetic teaching cases, without calendar dates or market data. The model uses a constant nonnegative effective annual rate, whole years and reinvested interest. It excludes cash flows, fees, taxes and intermediate cent rounding. There is no fitted parameter, backtest, bank-product comparison or empirical return claim.

Before implementation, acceptance required hand-calculated one- and multi-year values, zero rate, zero years, zero principal, invalid finite/nonfinite inputs and overflow. The test protocol also compares a separate year-by-year calculation, preserves C001's original displayed values, replays the demonstration twice and checks that substituting simple interest fails the tests.

Numeric comparisons require finite results and error at most $1e-12 * \max(1, \text{abs(expected)})$. This is an absolute tolerance below one unit and a relative tolerance for larger values. It is a test tolerance, not an accuracy guarantee for every possible input. Displayed output must match hand-authored text exactly. The complete project checks must pass before publication.

Mathematics and worked example

For a constant effective annual rate, $B_0 = P$ and $B_{(k+1)} = B_k * (1 + r)$. Repeated substitution gives $B_n = P * (1 + r)^n$. Total interest is $B_n - P$. Simple interest instead gives $S_n = P * (1 + n*r)$. At $n = 1$ both agree. At $n = 2$ the compound premium is $P*r^2$, because interest itself earns interest.

Scenario	Hand calculation	Balance (units)
One year at 5%	$1000 + 50$	1050.00
Two years compounded	$1050 + 52.50$	1102.50
Two years simple	$1000 + 2 * 50$	1100.00
Three years compounded	$1102.50 + 55.125$	1157.625
Zero rate, 40 years	$1000 * 1^{40}$	1000.00
Zero years, 5%	$1000 * 1.05^0$	1000.00

The two-year difference is $1000 * 0.05^2 = 2.50$ units. There is no rounding between years. The third year's 55.125 interest remains fractional; the demo prints that final balance with three decimals. A real account might round each posting according to its contract, producing a different calculation.

Five percent means 0.05, not 5.0. The latter is 500%: for a 2-unit principal and two years, $2 * 6 * 6 = 72$ units. The function accepts that synthetic rate because no upper economic bound is claimed. A valid numeric input does not imply a plausible financial assumption.

Function contract and C++ syntax

The declaration is `double compound_balance(double principal, double annual_rate, int years)`. The leading double names the return type. The parenthesised parameters describe the values it takes. `savings.hpp` shares this declaration with `main.cpp` and `test_savings.cpp`; `savings.cpp` contains the body. The build compiles and links the source files. `return balance` gives the result back without printing.

The function first rejects negative or nonfinite principal and rate, and negative years, using `std::invalid_argument`. Validation comes before shortcuts: a NaN rate remains invalid even when principal and years are zero. Valid zero principal, zero years or zero rate returns the principal. Otherwise `std::pow` computes the growth factor and multiplication produces the balance. A nonfinite factor or balance raises `std::overflow_error`. The tests catch the expected error type.

Years must be supplied as an integer. C++ can implicitly convert a fractional argument to `int` before entering this function, so the function cannot discover that original fraction. C003 must reject fractional textual horizons before conversion. Today's demonstration has fixed, correctly typed inputs.

Implementation and reproduction

cpp/projects/savings_growth now contains savings.hpp, savings.cpp and test_savings.cpp alongside the demonstration and hand-authored expected.txt. The demo calls the reusable function for one, two and three years. Its original one-year output is preserved, followed by a compound-versus-simple comparison. The Makefile tracks implementation and header dependencies and runs both C++ checks.

Run make cpp-savings to build and display the synthetic experiment. Run make cpp-check for 66 function checks and the exact output comparison. Run make check for the complete project. Run make -B cpp-check to force recompilation, and make note NOTE_DATE=2026-09-30 to recreate this PDF. No library or package dependency was added; products remain in the ignored build directory.

Observed results and validation

Check	Observed result
C++ function tests	66 checks pass: 14 value/zero, 19 invalid, 3 overflow, 30 yearwise
Synthetic experiment	1050.00, 1102.50, 1157.625 compound balances; 2.50 two-year premium
Reproduction	Two executable runs match hand-authored expected.txt bytes
Negative control	Simple-interest substitution fails the two-year test
Existing Python suite	All 363 tests pass; unchanged Python test count
Code quality	Ruff lint/format pass; strict mypy passes on 17 package files
Build and dependencies	C++17 warnings-as-errors build and pip consistency pass
Prior provenance	Q003 still verifies its four original source hashes

The thirty yearwise comparisons use $B_{\text{next}} = B + B \cdot r$, without `std::pow` in the reference. They supplement independent hand examples; they do not establish universal numerical accuracy. The harness checks finiteness explicitly so NaN cannot pass by making an error comparison false. It uses exit status and exceptions, so disabling C++ assertions does not disable these checks.

All implementation checks passed on their first run. Restricted network DNS required approved access for the repository fetch. No permission setting was weakened and no failure was suppressed. PDF verification and the final staged-file audit are recorded in the separate validation report.

Primary sources consulted

Microsoft's official C++ functions documentation explains declarations, parameters, definitions and returned values. These concepts support separating calculation from output.

<https://learn.microsoft.com/en-us/cpp/cpp/functions-cpp?view=msvc-170>

Microsoft's `pow` and `isfinite` references describe exponentiation, numeric range errors and finite-value classification. The financial convention is our stated teaching assumption, not a product recommendation.

<https://learn.microsoft.com/en-us/cpp/c-runtime-library/reference/pow-powf-powl?view=msvc-170>

<https://learn.microsoft.com/en-us/cpp/c-runtime-library/reference/finite-finitf?view=msvc-170>

Sources consulted on 30 September 2026. Local validation used Apple Clang 21.0.0 in C++17 mode.

Limitations and interview explanation

Binary floating-point approximates many decimals. A tiny positive r can make $1 + r$ round to 1; large horizons amplify approximation. Printing two decimals does not make the calculation an exact money ledger. Negative rates, fractional years, varying rates, inflation, cash flows, day counts and bank rounding are outside this lesson. No performance improvement or production suitability is claimed.

The direct power approach conservatively rejects a nonfinite intermediate factor. For example, with the smallest positive normal double principal, a 100% rate and 1100 years, the mathematical final balance can fit even though the factor 2^{1100} overflows double. This case deliberately raises. Log-domain scaling could extend the numeric range but requires a separate precision contract and tests.

For an interview, first state rate units and compounding frequency. Derive the recurrence and show why one-year tests cannot distinguish the two conventions. Explain zero cases and validation before shortcuts. Then discuss the difference between a tested floating-point approximation and contractual cash accounting. Repository output is evidence to study, not a claim of personal mastery.

Next work and necessary input

C002 is complete. C003 is Ready for Wednesday October 7: command-line parsing with clear units and malformed-input rejection. Q008 synthetic corporate-action returns is next on October 1. No input is needed for either independent task. Empirical research still requires lawful data entitlement/sample evidence under Q004a and the later Q005b integration work. Do not send credentials.

Sidiq review remains Not yet reviewed. Publication is authorised separately from personal review. The daily schedule remains 08:00 Europe/London, including daylight-saving changes. New research stops after 31 December 2026 pending the next programme scope.

Check your understanding

1. Why does the second year earn 52.50 units rather than 50?
2. What should a valid zero-year calculation return, and should a NaN rate be accepted then?
3. Does a rate input of 5.0 mean 5%, and what does the function assume about the frequency?
4. Why can the overflow guard reject a case with finite mathematical final wealth?
5. Does an int parameter guarantee that the caller never supplied a fractional expression?

Suggested answers

1. Interest stays in the balance: $1050 * 0.05 = 52.50$. The extra 2.50 is interest on prior interest. Simple interest uses only the opening 1000 each year and therefore misses this amount.
2. Return the principal because $(1 + r)^0 = 1$. Reject NaN first; zero years must not bypass the function's input contract. The same validation-first rule applies to zero principal and zero rate.
3. No: 5.0 means 500%, while 0.05 means 5%. The rate is effective per year with annual reinvestment, not a nominal rate divided into monthly periods or a continuously compounded rate.
4. The implementation evaluates the growth factor before multiplying by principal. That intermediate factor can exceed double's range even if a tiny principal would scale the final result back into range.
5. No. C++ can convert an expression before the call. The caller must supply whole-year integers; C003's parser must check the original text before converting it. No text parser exists today.